



Universidade do Minho
Licenciatura em Ciências da Computação

Unidade Curricular de Computação Gráfica

Ano Letivo de 2023/2024

Primitivas Gráficas – Fase 1

Grupo 12

Bruna Micaela Rodrigues Araújo a84914,

Hugo Alexandre Peres Ferreira a100082,

Nuno Filipe Norberto Gonçalves de Oliveira a53971

Março, 2024

CG

Data de Recepção	
Responsável	
Avaliação	
Observações	

Primitivas Gráficas – Fase 1

Grupo 12

Bruna Micaela Rodrigues Araújo a84914,

Hugo Alexandre Peres Ferreira a100082,

Nuno Filipe Norberto Gonçalves de Oliveira a53971

Março, 2024

Introdução

Este presente relatório é referente à primeira fase de um projeto desenvolvido no âmbito da unidade curricular de Computação Gráfica, que se centra no desenvolvimento de um mini motor gráfico 3D.

Esta fase requer o desenvolvimento de duas aplicações: a aplicação ***generator***, responsável por gerar ficheiros com as informações do modelo, mais especificamente os vértices necessários para desenhar primitivas gráficas; a aplicação ***engine*** que interpreta um ficheiro de configuração em XML para desenhar os respetivos modelos.

Para implementar estas aplicações, recorreu-se à ferramenta OpenGL e à linguagem de programação C++.

Ao longo deste relatório irão ser explicadas, de forma detalhada, todas as decisões tomadas.

Índice

Introdução	i
Índice	ii
Índice de Figuras	iii
1. Estrutura do Projeto	1
1.1. Contextualização	1
1.2. <i>Generator</i>	1
1.3. <i>Engine</i>	2
2. Primitivas Gráficas	3
2.1. Plano (<i>Plane</i>)	3
2.2. Caixa (<i>Box</i>)	4
2.3. Cone	4
2.4. <i>Esfera (Sphere)</i>	5
3. Resultados	7
3.1. Plano (<i>Plane</i>)	7
3.2. Caixa (<i>Box</i>)	7
3.3. Cone	8
3.4. <i>Esfera (Sphere)</i>	8
4. Conclusão	9

Índice de Figuras

Figura 1 – Exemplo de Vértices Guardados	2
Figura 2 – Geração de Triângulos	3
Figura 3 – Construção do Cone	5
Figura 4 – Construção da Esfera	6
Figura 5 – Plano (<i>Plane</i>) com <i>length</i> 2 e <i>divisions</i> 3	7
Figura 6 – Caixa (<i>Box</i>) com <i>length</i> 2 <i>divisions</i> 3	7
Figura 7 – Cone com <i>radius</i> 1 <i>height</i> 2 <i>slices</i> 4 <i>stacks</i> 3	8
Figura 8 – Cone com <i>radius</i> 1 <i>height</i> 2 <i>slices</i> 4 <i>stacks</i> 3	8
Figura 9 – Esfera (<i>Sphere</i>) com <i>radius</i> 1 <i>slices</i> 10 <i>stacks</i> 10	8

1. Estrutura do Projeto

1.1. Contextualização

Com o intuito de manter o código bem organizado e modular, o projeto foi estruturado da seguinte forma:

- Diretoria **engine**: contém o código necessário para a ler os ficheiros XML e para a visualização 3D dos modelos;
- Diretoria **generator**: contém o código responsável pelo cálculo dos vértices utilizados na criação dos modelos das várias primitivas gráficas e pela criação dos ficheiros “.3d”.

1.2. Generator

Cabe à aplicação **generator** calcular os vértices dos triângulos necessários para desenhar as diferentes primitivas gráficas e guardar essa informação num ficheiro “.3d”.

Nesse ficheiro, os vértices são guardados da seguinte forma: em cada linha guarda-se os valores correspondentes às coordenadas X, Y, Z de um vértice (3 *floats* separados por um espaço). Cada três vértices consecutivos formam um triângulo, tal como se mostra na *Figura 1* mais abaixo.

Esses ficheiros “.3d” serão posteriormente lidos pela aplicação **engine**.

Para construir as diferentes primitivas gráficas, devem ser fornecidos ao **generator** os seguintes argumentos:

- **Plane**: *length*, *divisions* e *filename*. Pode ser passado ainda um quarto argumento *front*, que é possível omitir, sendo o seu propósito auxiliar a definir para onde o **plane** está virado;

- **Box:** *length, divisions e filename;*
- **Sphere:** *radius, slices, stacks e filename;*
- **Cone:** *radius, height, slices, stacks e filename.*

No ponto **2.** serão abordados cada uma destas primitivas gráficas com alguma minúcia e atenção.

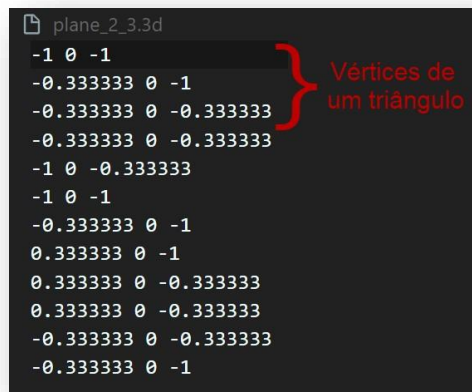


Figura 1 – Exemplo de Vértices guardados

1.3. Engine

Esta aplicação recebe um ficheiro de configuração escrito em XML, contendo as configurações da câmara e a indicação de quais os ficheiros a carregar.

O **engine** grava os dados recebidos em XML numa estrutura de dados criada de seu nome “*Config*”.

A câmara move-se na superfície de uma esfera, sempre a olhar para o seu centro, sendo o ponto $P = (px, py, pz)$ a posição da câmara.

Resta então calcular, como se mostra de seguida, os ângulos e o raio para ser possível a deslocação na superfície da esfera:

$$raio = \sqrt{px^2 + py^2 + pz^2}$$

$$beta = \sin^{-1}\left(\frac{py}{raio}\right)$$

$$alpha = \cos^{-1}\left(\frac{pz}{raio * \cos(beta)}\right)$$

2. Primitivas Gráficas

2.1. Plano (*Plane*)

O plano é um quadrado de comprimento *length*, desenhado no plano XZ e centrado na origem (0, 0, 0), subdividido em divisões (*divisions*) ao longo dos eixos X e Z, o que origina $divisions \times divisions$ divisões, ou seja, $divisions \times divisions \times 2$ triângulos.

De forma a serem calculadas as coordenadas dos vértices de cada triângulo, foram criadas as variáveis *px*, *py*, *pz*, *mid* e *increment*, sendo que *px*, *py* e *pz* dizem respeito às coordenadas X, Y, Z de cada vértice, *mid* é o valor do ponto médio do comprimento (*length*) e *increment* representa o comprimento de cada divisão.

Para cada divisão são criados quatro vértices, que são utilizados para originar os triângulos. Foi usada a regra da mão direita para saber qual das faces do triângulo ficará visível.

Contruiu-se o plano desenhando triângulos da esquerda para a direita e de baixo para cima, tal como ilustra a *Figura 2* onde foram gerados os vértices da divisão 1, composta pelos triângulos originados pelos vértices (*p1*, *p2*, *p3*) e (*p1*, *p3*, *p4*) e prosseguiu-se para as seguintes divisões por ordem numérica ascendente.

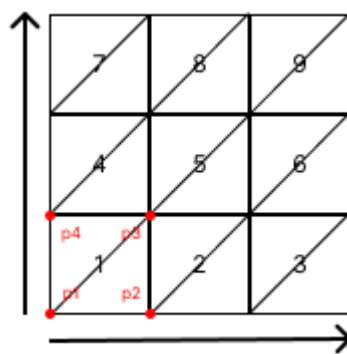


Figura 2 - Geração de Triângulos

2.2. Caixa (Box)

A caixa está centrada na origem e consiste na junção de seis planos. São criados dois planos paralelos em cada um dos planos XY, XZ e YZ, com distância *length* entre cada um, centrados em (0, 0, 0).

O raciocínio adotado foi semelhante ao utilizado na implementação da função *plane*.

De forma a saber qual a orientação do plano, utilizou-se a flag *front*.

2.3. Cone

O cone recebe como argumentos o raio da base (*radius*), a altura do cone (*height*), o número de fatias (*slices*) e o número de camadas (*stacks*), tendo a sua base no plano XZ. De forma a calcular os ângulos, raios e alturas necessárias em cada fase da construção desta primitiva, criaram-se as seguintes variáveis: *alphaInc*, *heightInc*, *RadiusDec*, *currentHeight* e *currentRadius*.

alphaInc representa a variação de ângulo, valor necessário para o cálculo dos vértices dos triângulos do cone, tanto na construção da base, como das faces laterais, através das coordenadas cartesianas. É calculado a partir do perímetro da circunferência a dividir pelo número de *slices*.

$$\alphaInc = \frac{2\pi}{slices}$$

heightInc representa a diferença de alturas entre cada camada. É calculado a dividindo a altura do cone a pelo número de *stacks*.

$$heightInc = \frac{height}{stacks}$$

radiusDec representa o valor que se deve subtrair ao raio da base à medida que a altura do cone vai aumentando. É calculado dividindo o raio pelo número de *stacks*.

$$radiusDec = \frac{radius}{stacks}$$

As variáveis *currentHeight* e *currentRadius* representam a altura e raio, respetivamente, naquele determinado momento. Considerando *i* como a *stack* para a qual se está a calcular os valores dos vértices dos triângulos, os valores das variáveis são atualizados da seguinte forma:

$$currentHeight = heightInc * i$$

$$currentRadius = radius - (radiusDec * i)$$

De referir que na última *stack* só há a necessidade de desenhar um triângulo por *slice* e não um quadrado completo.

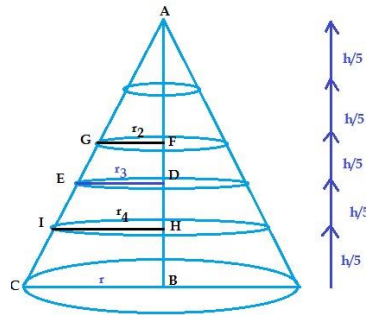


Figura 3 - Construção do Cone

2.4. Esfera (Sphere)

A esfera está centrada na origem, tal como a caixa, e para o cálculo dos vértices de cada triângulo recorreremos às variáveis *alpha*, *beta*, *alphaInc* e *betaInc*.

Os valores iniciais dos ângulos onde se vai começar a desenhar a esfera são representados por *alpha* e *beta*, inicializados a 0 e $-\pi/2$ respetivamente.

alphaInc e *betaInc* representam os valores da variação dos ângulos *alpha* e *beta*, calculados da seguinte forma:

$$alphaInc = 2\pi/slices$$

$$betaInc = \pi/stacks$$

Para determinar os pontos necessários a fim de desenhar a esfera, calcularam-se as coordenadas cartesianas dos quatro pontos resultantes de cada intercessão de uma *slice* com uma *stack*.

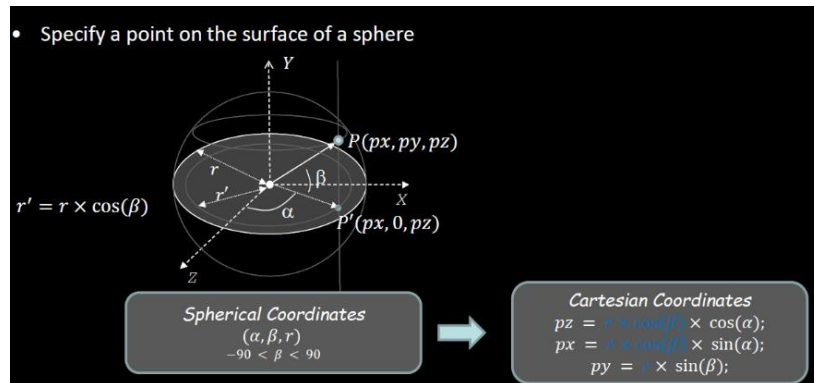


Figura 4 - Construção da Esfera [Slides: Drawing a Cylinder, Practical Classes]

3. Resultados

3.1. Plano (Plane)

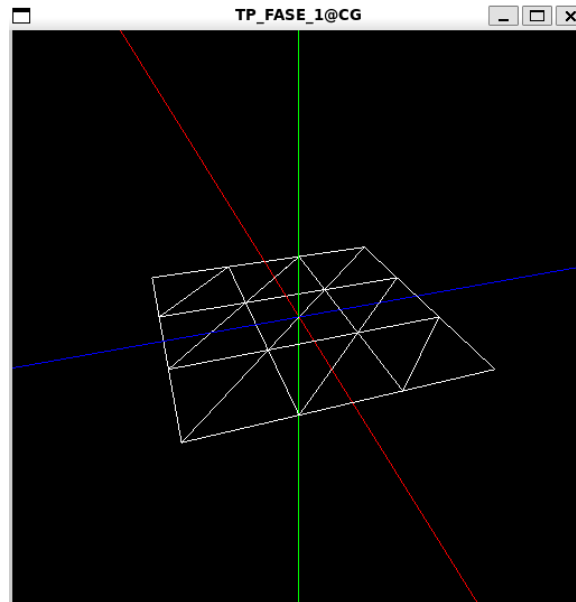


Figura 5 - Plano (*Plane*) com *length* 2 e *divisions* 3

3.2. Caixa (Box)

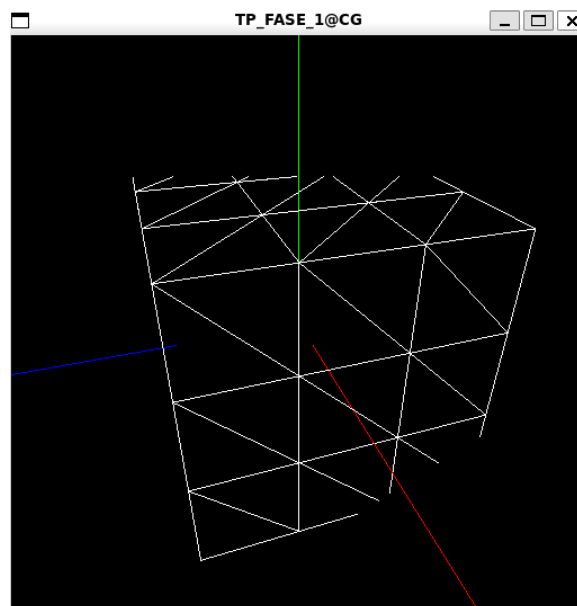
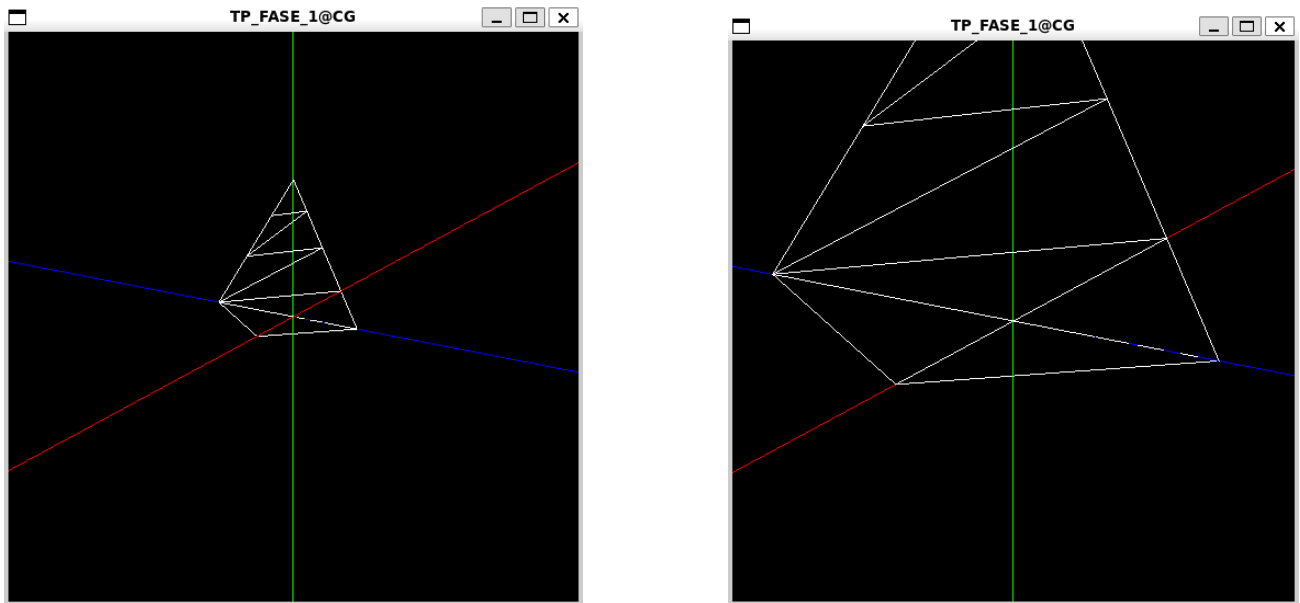


Figura 6 - Caixa (*Box*) com *length* 2 *divisions* 3

3.3. Cone



Figuras 7 e 8 - Cone com *radius 1 height 2 slices 4 stacks 3*

3.4. Esfera (Sphere)

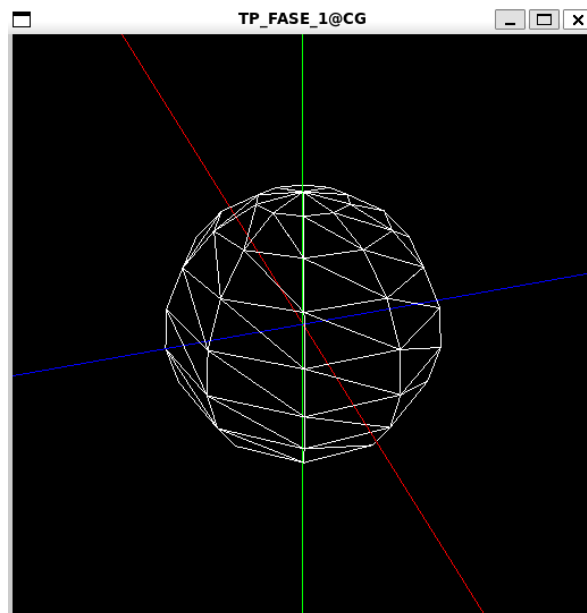


Figura 9 - Esfera (Sphere) com *radius 1 slices 10 stacks 10*

4. Conclusão

O presente relatório detalha a estrutura e o funcionamento do projeto, enfatizando a organização modular e a eficiência na geração e visualização de modelos gráficos em 3D.

A aplicação ***generator*** é responsável pela geração dos vértices necessários para criar diversas primitivas gráficas, armazenando-os em arquivos ".3d".

Por sua vez, a aplicação ***engine*** recebe esses dados através de um arquivo XML de configuração, possibilitando a visualização dos modelos em 3D.

Em suma, o projeto apresenta de forma bem definida tudo o que nos foi pedido, para além de uma implementação sólida, capaz de atender às necessidades de geração e visualização de modelos gráficos em 3D de forma eficiente e precisa.